Trajets en bus

1 Préliminaires

1.1 Récupération des fichiers

Depuis la ligne de commande, naviguez vers un répertoire vous appartenant, et exécutez la commande suivante, qui téléchargera et décompressera un répertoire nommé bus :

```
curl cdn.sci-phy.org/mp2i/tp20-lignes_bus.tgz | tar xvz
```

On y trouvera un fichier source `bus.c` à compléter, un module `heapq`, un makefile, et des fichiers contenant les données d'un service de transport en commun routier.

1.2 Présentation du problème

Durant cette séance, nous allons nous efforcer de mettre en place un outil de planification d'itinéraires pour un service de transports. Le fichier « `stops.dat` » contient la liste des cent deux arrêts. Chaque arrêt est défini par un identifiant (entier positif entre 0 et 101), son nom, sa longitude, sa latitude, et un booléen indiquant s'il s'agit d'un arrêt « de transfert » où plusieurs lignes se retrouvent.

id	lat	long	transfer	name
0	41.2318	10.2192	false	Archimedes
1	41.23162	10.2207	false	Euclide
2	41.23144	10.22208	true	Kepler
3	41.23045	10.2228	true	Lagrange
4	41.22955	10.22352	true	Poincaré
5	41.22955	10.2246	false	Hilbert
6	41.22955	10.2258	true	Gauss
...				

Dans le programme, on mémorise les arrêts dans un tableau de structures définies par :

```c
struct stop {
    char* name;
    double lat, lng;
    bool transfer;
}
typedef struct stop Stop;
```

L'arrêt d'identifiant i sera rangé dans la case i du tableau. La fonction `load_stops` se charge de lire le fichier, puis d'allouer et remplir le tableau, et renseigne aussi une variable nommée `nb_stops` contenant le nombre d'arrêts (soit la taille du tableau).

Un second fichier « `times.dat` », chargé par la fonction `load_times`, contient la liste des instants de passage des bus à un arrêt. On y récupère trois informations : `stop` correspond à l'identifiant de l'arrêt, `time` l'instant du départ, en secondes depuis $0h$. Enfin, `route` est un identifiant d'un bus sur une ligne : l'identifiant est le même pour tous les arrêts effectués par un même bus de son point de départ, à une extrémité de la ligne, à sa destination à l'autre extrémité. Cet identifiant change lorsque le bus repart en sens inverse.

route	stop	time
11260	0	22020
11260	1	22094
11260	2	22166
...		
11260	17	23234
11260	18	23301
11260	19	23375
11297	0	23700
11297	1	23773
11297	2	23844
...		

route	stop	time
34644	26	66027
34644	27	66095
34644	28	66161
34644	29	66238
34644	9	66348
34644	10	66439
34644	30	66536
34644	31	66621
34644	32	66692
34644	33	66756
...		

route	stop	time
46706	56	59914
46706	99	59983
46706	100	60050
46706	101	60110
46743	93	60640
46743	94	60702
46743	95	60772
46743	96	60829
46743	32	60880
46743	97	60955
...		

Les moments de passage des bus à un arrêt donné sont mémorisés dans un tableau de structures définies par :

```c
struct time {
    int route, stop, time;
    int n_route, n_stop, n_transfer;
};
typedef struct time Time;
```

Pour naviger entre les différentes lignes, la structure contient trois informations supplémentaires, qui ne sont pas chargées depuis le fichier mais qu'il va nous falloir renseigner :

- `n_route` contient le numéro de la case, dans le tableau, correspondant à l'arrêt suivant du bus sur la route, et -1 s'il n'y en a pas (si l'arrêt correspond au terminus du bus) ;
- `n_stop` correspond au numéro de la case, dans le tableau, du prochain passage d'un bus au même arrêt, et -1 s'il s'agissait du dernier passage ; on supposera (c'est le cas pour les données) qu'il n'y a pas deux passages dans la même seconde au même arrêt ;
- `n_transfer` sera décrit ultérieurement.

2 Mise en place du graphe

2.1 Reconstruction des routes

Les champs `n_route` et `n_stop` n'étant pas renseignés par la fonction de chargement. La première tâche consistera donc à remplir ces champs pour chacune des cases du tableau `times`. La taille du tableau est mémorisée dans la variable `nb_times`.

Pour remplir ces informations, et pour toutes les fois où cela pourra être utile, on précise que les « routes » des bus sont rangées consécutivement, dans l'ordre, dans le tableau. Autrement dit, le champ `n_route` dans la case `i` du tableau `times` contenant les arrêts contiendra nécessairement soit `i+1`, soit -1.

1. Compléter la fonction `build_graph` pour renseigner les champs `n_route` dans chaque case du tableau.

2. Proposer une fonction `terminus` prenant en argument un identifiant de passage (un numéro de case dans le tableau `times`) et retournant l'identifiant du passage au terminus du bus sur sa route actuelle (à nouveau, un numéro de case dans le même tableau).

3. Compléter la fonction `destination` prenant, de la même façon, un identifiant de passage, et affiche le nom du terminus, le nombre d'arrêts jusqu'à celui-ci (on affichera 0 si l'arrêt en question est le terminus) et le temps nécessaire à rejoindre le terminus.

Pour afficher l'heure sous le format habituel, si `time` désigne l'instant en secondes, on pourra utiliser une variante de

```
printf("Passage à l'arrêt à %dh%02d !", (times/3600)%24, (times/60)%60);
```

L'utilisation d'un reste de division par 24 pour les heures permettrait d'afficher correctement d'éventuelles heures de passage après minuit.

4. Tester la fonction avec les passages `0`, `29` et `59`.

2.2 Reconstruction des passages à un arrêt

5. Pour les champs `n_stop`, il n'y a aucune règle dans la table, il faut donc chercher, parmi tous les passages renseignés pour ce même arrêt, celui qui le suit immédiatement en terme d'heure. Compléter la fonction `build_graph` pour renseigner l'ensemble des champs `n_stop` de la façon qui vous conviendra (un algorithme en temps quadratique conviendra, on pourra réfléchir à une solution plus efficace, mais il n'est pas demandé de l'implémenter).

6. Compléter la fonction `next_departure` qui, pour un arrêt et une heure donnée, retourne l'index de la case dans le tableau `times` correspondant au passage d'un bus à cet arrêt le plus tôt possible après l'heure indiquée, et -1 s'il n'y en a aucun. On pourra effectuer une recherche simple dans tout le tableau.

7. Compléter la fonction `next_five_departures` qui, pour un arrêt et une heure donnée, affiche les cinq prochains départs avec, pour chaque départ, l'heure de départ, la destination (terminus) et l'heure d'arrivée au terminus. S'il reste moins de cinq départs, on réduira évidemment cette liste. On utilisera la fonction précédente pour le premier départ, et la structure intelligemment pour les autres.

8. Afficher les prochains départs de l'arrêt 57 (Léonard Euler) à $12h37$.

2.3 Contrôle du graphe

9. Compléter la fonction `all_destinations` qui, pour un identifiant d'arrêt, affiche toutes les destinations (terminus) possibles pour les bus partant de cet arrêt.

10. Tester la fonction précédente pour les arrêts 19 (Alan Turing) et 26 (Isaac Newton).

11. Écrire une fonction `reachable` prenant en argument un arrêt et une heure, et affichant le nombre, suivi de la liste des noms de tous les arrêts que l'on peut atteindre (chaque arrêt ne devant apparaître qu'une seule fois).

12. Déterminer les arrêts que l'on peut atteindre en partant de l'arrêt 18 (Boole) à 22h40.

3 Plus courts chemins

3.1 Première implémentation

13. Proposer une fonction `Dijkstra` prenant en argument un index de case dans le tableau `times` représentant un arrêt et une heure de départ, et un entier représentant un arrêt que l'on souhaite rejoindre, déterminant un plus court chemin avec l'algorithme de Dijkstra et retournant un pointeur vers un tableau alloué par la fonction `Dijkstra`, contenant la liste des index dans le tableau `times` correspondant à ce chemin, **en terminant ce tableau par la sentinelle -1**. Si aucun chemin n'existe, on retournera `NULL`

Pour aider à l'écriture de cette fonction, on fournit un module `heapq` qui fournit une file de priorité.

14. Tester l'algorithme avec la fonction `disp_route` qui prend en argument un identifiant de station de départ, une heure et une station d'arrivée, et affiche de façon élémentaire la route suivie.

3.2 Amélioration

Un voyageur n'a jamais aucun intérêt de descendre à un arrêt qui n'est pas sa destination si cet arrêt ne permet pas de changer de ligne. Pour accélérer la recherche dans le graphe, on souhaite donc ajouter des liens, pour chaque sommet du graphe, vers le prochain arrêt de transert, s'il existe. C'est la raison de l'existence des champs `n_transfer`

15. Compléter une dernière fois la fonction `build_graph` pour renseigner les champs

`n_transfer` (comme précédemment, on mettra -1 s'il n'y a pas de station de transert en aval sur la route).

16. Modifier la fonction `Dijkstra` pour qu'elle tienne compte de ces informations.

17. Le temps de calcul est-il modifié ? Qu'y gagne-t-on ? Comment pourrait-on gagner du temps de calculs (on ne demande pas d'implémenter une telle solution).

18. Modifier la fonction `disp_route` pour qu'elle indique de façon plus claire la route : on affichera les montées dans un bus, avec l'arrêt, l'heure et la destination du bus, et les descentes, avec arrêt et heure.

3.3 Recherche efficace

Pour accélérer la recherche, on peut souhaiter utiliser une heuristique. Pour ce faire, on commence par déterminer le temps le plus court qui permette d'aller d'un arrêt A à un arrêt B *en une seule étape*. Ces informations seront regroupées dans une grande matrice, représentée en C comme un tableau d'entiers à *une seule dimension* de taille `nb_stops * nb_stops`. Le temps minimal pour aller d'un arrêt `i` à un arrêt `j` sera rangé dans la case `i*nb_stops + j`. On utilisera `MAX_INT` lorsqu'il n'y a pas de connexion directe (on pourra mettre des `MAX_INT` sur la diagonale).

19. Proposer une fonction `gen_costs` qui alloue, construit et retourne un tel tableau.

20. En utilisant l'algorithme de Floyd-Warshall, écrire une fonction qui retourne la somme minimale de tous les déplacements pour aller d'un point A à un point B (ce ne sont PAS des temps de parcours, on néglige les temps pour les changement, et on utilise le temps le plus court entre deux arrêts sans se préoccuper du moment où ils sont possibles, mais il s'agit d'une sous-estimation de ces temps de parcours).

21. Utiliser ce tableau et l'algorithme A* pour proposer une alternative plus efficace à l'algorithme de Dijstra.